

Lab 3 Report

Ryan Wilson

Due: 02/10/2026 @ 11:00 AM

1 Exercise 5:

- 1. $123 + 567 = 690$

Expected behavior: 123 plus 567 should be 690.

Compiler errors: The compiler reports no error, but the output is not correct.

Reason: The variable is set as an `unsigned char`, so which has a maximum upper storage limit of 255. The variable overflows due to this and outputs a different number.

Corrections: I would correct this by using a larger integer type such as `int` or `unsigned int`.

- 2. $1/10 = 0.1$

Expected behavior: $1/10$ should be 0.1.

Compiler errors: Compiler says float is not able to be divided by an integer, so the result prints 0.

Reason: Float variables are not compatible in math equations with whole integers.

Corrections: I would change all `unsigned int` or `int` variables to that of `float` as well as change the `%d` to a `%f` in the `printf` prompt to display a float. This gets the final answer of 0.1.

- 3. $3.6 = 3.60000000000000000000$

Expected behavior: Printing 3.6 with 20 decimal places would give exactly one 6 and 19 zeros.

Compiler errors: No errors given by compiler, but the last 4 digits of the output are not zeros.

Reason: It does not show up as exactly `3.60000000000000000000` because floating-point numbers using binary cannot be accurately displayed to 20 digits.

Corrections: This is not a possible fix after speaking with TAs.

2 Reference sheet:

Command	Option	Description of output and interaction
<code>gcc</code>	<code>-o</code>	<code>gcc</code> compiles the script and <code>-o</code> assigns it to an output file named after the option.
<code>isalpha()</code>		Checks if a character is an alphabetic letter.
<code>scanf()</code>		Reads a formatted input from the standard input (<code>stdin</code>).
<code>sscanf()</code>		Reads a formatted input from a string and stores it in a variable.
<code>time</code>		Measures the execution time of a program.
<code>fgets()</code>		Reads an entire line of an input including white spaces.
<code>printf()</code>		Prints a formatted output.
<code>sizeof()</code>		Returns the size (bytes) of a variable.
<code>getchar()</code>		Reads a single character from an input. I used it to clear previous lines of an input.
<code>long long int</code>		Gets rid of error for integer overflow, allowing for super large integer values to be used.
<code>i++</code>		Post-increment command that tells the counter “i” to move to the next iteration by 1. Acts like <code>i = i + 1</code> .

3 Exercise 10:

Performance:

When running each program from $n = 1$ to $n = 100,000$ to $n = 1,000,000,000$ I found that BruteForceAdder had a longer and longer runtime as n increased, but that GaussAdder's runtime did not change as significantly when running larger n values.

Description of Brute Force algorithm:

The brute force version of summation simply just adds all integers from 1 to n . For example, $n = 5$ would compute $1 + 2 + 3 + 4 + 5$ and output 15. The Big-O for brute force's time complexity classification is $\mathcal{O}(n)$, meaning that the loop loops through each value of n once linearly.

Description of Gauss algorithm:

The Gauss version of summation computes the sum using a constant number of operations, being completely independent of the variable n . This means that the Big-O for Gauss' time complexity classification is $\mathcal{O}(1)$, making it *significantly* faster at larger n values than the brute force method. Having an intelligent algorithm rather than a brute force one makes a substantial difference in decreasing processing time for very large values of n .

n	Brute Force Time	Gauss Adder Time
1	0.002s	0.002s
10	0.002s	0.002s
100	0.002s	0.002s
1000	0.002s	0.002s
100000	0.002s	0.002s

Sources:

<https://www.geeksforgeeks.org/c/c-for-loop/>
<https://www.geeksforgeeks.org/c/c-program-to-find-sum-of-elements-in-a-given-array/>
<https://www.geeksforgeeks.org/cpp/difference-between-long-int-and-long-long-int-in-c-cpp/>
<https://www.geeksforgeeks.org/c/whats-difference-between-and/>
<https://www.geeksforgeeks.org/c/tokens-in-c/>
<https://www.geeksforgeeks.org/c/how-to-read-data-using-sscanf-in-c/>
<https://www.geeksforgeeks.org/c/scanf-in-c/>
<https://www.geeksforgeeks.org/c/scanfns-str-vs-getsstr-in-c-with-examples/>
<https://www.geeksforgeeks.org/c/c-loops/>
<https://www.geeksforgeeks.org/c/increment-and-decrement-operators-in-c/>
<https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/>